Machine Learning techniques for malicious PDF detection

Preprint, compiled January 9, 2023

Lorenzo Ippolito, Mattia Rosso, Martino Picasso

Abstract

This project presents a thorough analysis of machine learning techniques for detecting malicious PDF files. By comparing the performance of three binary classification models - K-Nearest Neighbors (KNN), Support Vector Machines (SVM), and Decision Trees - we demonstrate the effectiveness of these approaches in achieving high accuracy and low false negative rates. What makes this project stand out is its focus on the machine learning models themselves, including the careful selection of hyperparameters to optimize their performance. Additionally, the inclusion of KNN as a valid model is rare in the field, and the analysis of evasion techniques adds another layer of depth to the study. Overall, this paper offers a comprehensive look at the use of machine learning for malicious PDF detection and the potential vulnerabilities of these models.

1 Introduction

PDF (Portable Document Format) is a file format used for exchanging and sharing documents. It can contain various types of content, including text, images, multimedia, and executable code. Keywords, also known as actions or triggers, are used to specify different types of content in PDF files and can be used to detect malicious content. In addition to keywords, PDF files may contain other elements such as fonts, annotations, and form fields, which can be exploited for malicious purposes. The structure of a PDF file is important for detecting malicious content and analyzing keywords and other elements can provide insight into the contents of the file and whether it may be malicious.

2 Related work

This dataset has been cited in many publications on the topic of malicious PDF detection, and it provides a balanced number of both malicious and benign PDFs. By using this dataset, we can compare our results to those reported in previous publications and see if our models are performing better or worse. See Section 2.

2.1 Related work: second level

2.1.1 Related work: third level

3 Datasets and features

As the first step in building our model, we needed to obtain a dataset of PDFs to use for training and evaluation and we decided to use the Contagio dataset. It is a collection of malicious PDF files that has been widely used for research and testing purposes. The dataset was created by the security firm Mandiant and contains about 20000 PDF samples, including malware, phishing attacks, and other types of malicious content. The samples in the dataset are representative of the types of threats that are commonly encountered in the wild. Next, we needed to extract the data from the PDFs in order to use it for training and evaluation. We chose to perform a static analysis, which does not require opening the PDFs in a reader. Indeed, we used a tool called PDFId to extract relevant information from the PDFs. PDFId characterizes each PDF by extracting the number of occurrences of specific keywords. This allows us to map each PDF into an array of 21 elements, with each element representing the count of a specific keyword. We are therefore working with samples of 21 integer features. Among the keywords that we selected, some are related to the structure of the PDF file, such as obj and endobj, which mark the beginning and end of an object, and stream and endstream, which mark the beginning and end of a stream of data. We also included xref, trailer, and startxref, which pertain to the organization of the PDF file. Next, there are features related to the content of the PDF file: we included /Page, which specifies a page object in the PDF file, and /Encrypt. We also included /ObjStm, which denotes an object stream, and /JS and /JavaScript, which indicate the presence of JavaScript code. Additionally, we included features related to actions that can be triggered by the PDF file, such as /AA, /OpenAction, /XFA and /AcroForm, that could potentially exfiltrate information from users. We also included the features related to the types of data that may be present in the PDF file, such as /JBIG2Decode, /RichMedia, /Launch and /EmbeddedFile. Finally we also chose /Colors, which specifies the presence of more than 16 million colors

The output of PDFId is then converted into a CSV file, which can be easily accessed using the numpy and pandas libraries. This CSV file serves as the basis for our data extraction pipeline, which we have implemented using the functions provided by PDFId. It is important to note that PDFId is a tool specifically designed for the analysis of malicious PDFs, as the 22 keywords it extracts are well known to be discriminant for PDF malware analysis. These keywords do not represent the full set of possible keywords that may appear in a PDF file. For our classification task, we will be performing a binary classification, with malicious PDFs being labelled as positive samples and benign PDFs being labelled as negative samples. This allows us to easily train and evaluate our models, as we only need to predict whether a given PDF is malicious or not.

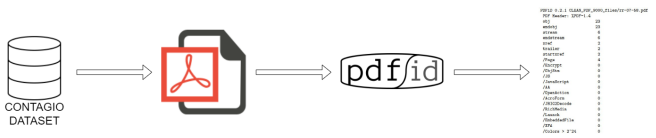

Figure 1: Features extraction pipeline

*correspondence: email@institution.edu

Publication doi

Preprint doi

Commenti

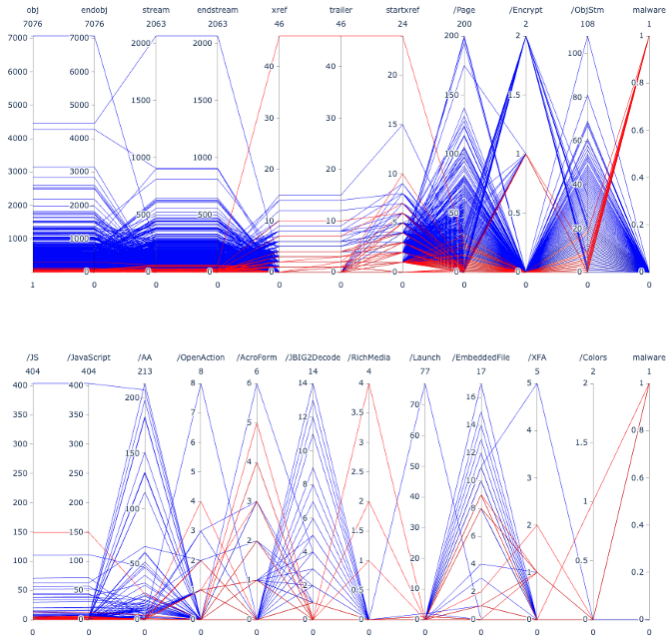

Figure 2: The plots show how the accuracy (left) and the FNR (right) change when the value of C hyperparameter changes

3.0.1 Model validation

We statically split the dataset in train and test. The training set, which consists of 80% of the samples, is used to train and validate the models using a k-fold cross validation approach. The test set, which consists of 20% of the samples, is not used to make decisions during the model building phase, but is instead used to make final considerations about the performance of the models trained with the best parameters/hyperparameters. To evaluate the performance of our models, we used two measures: the False Negative Rate (FNR) and Accuracy. The FNR is defined as

$$FNR = \frac{FN}{FN+TP}$$

We aimed to minimize the FNR as much as possible, as False Negatives can be particularly damaging in the context of malicious PDF detection. False Negatives can lead to the failure to detect and prevent attacks and can have serious consequences for individuals and organizations. At the same time, we also took into consideration the value of Accuracy, as the penalization of False Negative errors should not result in an excessively high number of False Positive errors. It is important to find a balance between minimizing False Negatives and minimizing False Positives, as both can have negative consequences. False Positives can lead to unnecessary alerts and can waste resources and can also erode trust in the system.

4 SUPPORT VECTOR MACHINES

Support Vector Machines are linear classifiers that look for maximum margin separation hyperplanes. The primal formulation of the soft-margin SVM problem consists in minimizing the function:

$$J(w, w_0) = \frac{1}{2}||w||^2 + C\sum_{i=1}^{n}\xi_i$$
$$\text{s.t. } z_i(w^T x_i + w_0) \geq 1 - \xi_i$$
$$\text{and } \xi_i \geq 0$$

To define the hard margin SVM problem the penalty term $C\sum_{i=1}^{n}\xi_i$ has been added to the soft margin SVM problem. ξ_i are slack variables and estimate the number of missclassified samples. The hyperparameter C is meant to define the strongness of this penalization. We also introduce the dual problem that, besides the fact that is the one that *sklearn* optimizes, is also useful to explain how the adversarial samples will be crafted in section 8.

$$J_D(\alpha) = \sum_{i=1}^{n}\alpha_i - \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n}\alpha_i\alpha_j z_i z_j x_i^T x_j$$
$$\text{s.t. } 0 \leq \alpha_i \leq C$$
$$\text{and } \sum_{i=1}^{n}\alpha_i z_i = 0$$

The function J_D has to be maximized and a set of n Lagrangian multipliers α has been introduced to embed the primal constraints.

4.1 Linear SVM

We implemented a linear SVM classifier using the *sklearn* library. The samples x_i are features vectors of 21 elements (the one extracted from PDFiD).

4.1.1 Experiments

We have cross-validated the linear model by trying 10 different values for the C hyperparameter in the range $[10^{-6}, 10]$. As explained in section 3 we optimized for the False Negative Rate (FNR) with the constraint of obtaining a high accuracy anyway.

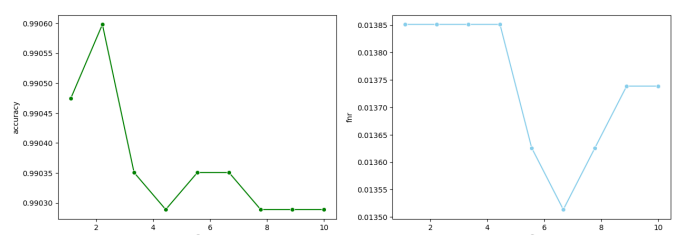

Figure 3: The plots show how the accuracy (left) and the FNR (right) change when the value of C hyperparameter changes

The plot in figure 3 show that the variations in terms of Accuracy and FNR when different values of C are used to train the linear SVM are not that evident. Here the results obtained with $C = 10^{-6}$ where not reported because of the significatively worse results (88.67% of accuracy and 2.94% of FNR).
Summarizing the best results we obtained:

C	Accuracy	FNR
2.22	99.06%	1.38%
6.67	99.03%	1.35%

Optimizing for the FNR the hyperparameter we have selected is $C = 6.67$.

4.2 *Kernel SVM*

What it makes the SVMs particularly powerful is the way how the dual problem is defined. We are indeed able to embed a non-linear decision function that computes the scalar product between x_i and x_j in a higher dimensional space without needing to explicitly compute the features expansion. A kernel function is defined as

$$k(x, y) = \Phi(x)^T \Phi(y)$$

And the J_D function is modified as:

$$J_D(\alpha) = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j z_i z_j k(x_i, x_j)$$

Also the inference step is modified accordingly:

$$y_t = w^T x_t + w_0 = \sum_{i=1}^{n} \alpha_i z_i k(x_t, x_i)$$

We have implemented a Radial Basis Function (RBF) kernel that is defined as:

$$k(x, y) = e^{-\gamma \|x-y\|^2}$$

4.2.1 *Experiments*

The cross-validation of the model has now to take into account two different hyperparameters (γ for the RBF kernel and C for the dual SVM problem). This time we exploited a grid search k-fold cross validation that makes use of the cross validation to train the model with each possible combinations of hyperparameters. The hyperparameters values we tried are:

- γ: 10 values in $[10^{-6}, 1]$
- C: 10 values in $[10^{-6}, 10]$

We now investigate the influence of both hyperparameters. What it's not displayed and analyzed in the plots is what happens with $C = 10^{-6}$ and $\gamma = 10^{-6}$, parameters under which the classifier performs a very poorly in terms of accuracy (around 54%) despite achieving a FPR equal to 0%. However, this is not better than a dummy classifier that safely detects every PDF as malicious so it's not of interest in this project.

In figure 4 it's clear that smaller values of γ lead to obtain better results both in terms of Accuracy and FNR. The γ parameter of the RBF kernel controls the width of the kernel, it determines how far the influence of a single training example reaches, and how smooth the decision boundary is. A lower value of gamma means a larger, less concentrated kernel, which can lead to smoother decision boundaries.

We also investigated the role of the C hyperparameter.

As we can observe in figure 5 the value of the C hyperparameter has less influence on the model than γ. The C hyperparameter controls the trade-off between the simplicity of the model (low bias, high variance) and the training error (low variance, high bias). A higher value of C means a higher penalty for misclassification while a lower value of C means a lower penalty for misclassification.
When the value of γ is sufficiently small the accuracy reaches the best values and the same happens for the FNR. The choice of C has clearly less impact on the performances.

Overall the best values we have found from cross validation are:

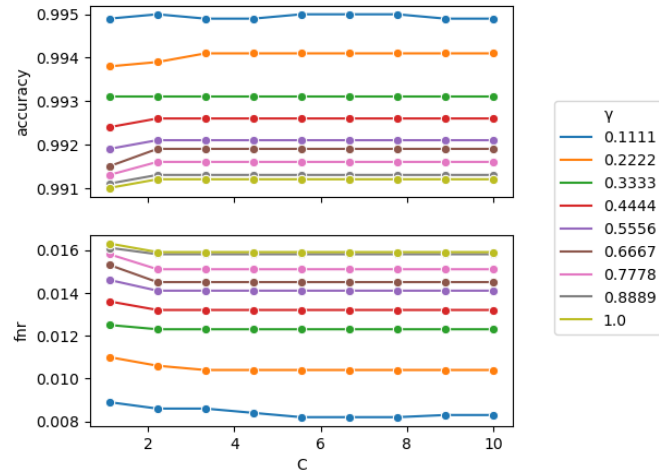

Figure 4: Accuracy (top) and FNR (bottom) when γ (colored lines) changes and C changes

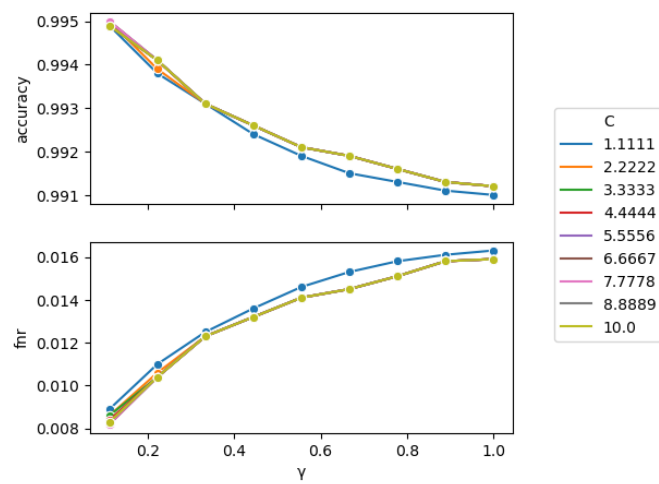

Figure 5: Accuracy (top) and FNR (bottom) when C (colored lines) changes and γ changes

C	γ	Accuracy	FNR
5.56	0.11	99.50%	0.82%

4.3 *Performances on the test set*

Finally, given the set of best hyperparameters found through cross validation on the entire training set we are going to asses the performances achieved on the test set.

4.3.1 *Linear SVM*

C	Accuracy	FNR
6.67	99.06%	1.58%

Table 1: Linear SVM - Performances on the test set

4.3.2 Kernel SVM

Secondly we train a RBF SVM on the entire training set and we do the predictions on the entire testing set.

C	γ	Accuracy	FNR
5.56	0.11	99.50%	0.82%

Table 2: RBF SVM - Performances on the test set

5 DECISION TREE AND RANDOM FOREST

The decision tree is a popular classification technique in which predictions are made in a sequence of single-attribute tests.

Decision trees are used to evaluate the expected values or expected utility of competing alternatives. Each internal node in a decision tree represents a test on an attribute, and each branch represents the outcome of the test. The paths from the root to the leaf nodes represent classification rules, and the leaf nodes represent class labels. In this test, we evaluated the performance of a decision tree using various parameters such as tree depth, minimum number of samples per leaf, and maximum number of features. We found that adjusting these parameters had a significant impact on the false negative rate and the accuracy of the model. For example, as shown in Figure 1, we found that the optimal tree depth parameter was 15, which provided the best balance between model performance and overfitting.

5.0.1 Experiments

5.1 Random forest

Random Forest is a machine learning algorithm that is used for both classification and regression tasks. It is an ensemble method, which means that it combines the predictions of multiple decision tree models to make a more accurate and stable prediction. To train a Random Forest model, multiple decision trees are trained on different samples of the data and their predictions are combined. This helps to reduce overfitting and improve the model's generalization ability. The algorithm is also robust to missing values, making it a versatile and widely used machine learning tool. In our analysis we chose this model because of its high accuracy and stability. It is known for performing well on a wide range of datasets and tasks. Also with Random Forest, we evaluated the performance varying parameters and in Figure we can see the results for maximum trees depth.

5.1.1 Experiments

6 KNN

K-Nearest Neighbors (KNN) is a simple yet powerful classification algorithm. It works by identifying the K number of training samples that are closest in distance to the input sample, and then assigning the input sample to the class of the majority of the K nearest neighbors. One of the key advantages of the KNN algorithm is its simplicity, as it requires little to no training to make predictions on new data. This is useful for our task, where the dataset is large and diverse and the goal is to identify patterns in the data.

6.1 Experiments

We performed a k-fold cross-validation with 5 folds. To determine the optimal value of k (the number of nearest neighbors to consider), we conducted a grid search over a range of possible values ($k \in [1, 8]$). The results of this search shows, as expected,

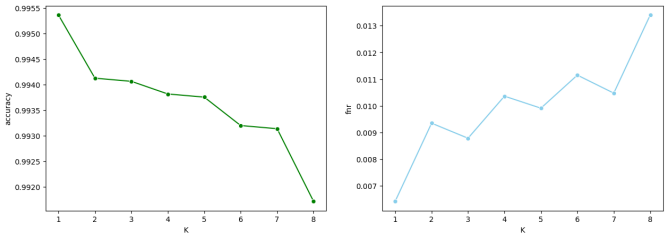

Figure 6: Accuracy (left) and FNR (right) when K changes

taht the best performance was achieved with k=1. This is due to the fact that when k is set to a small value, the model is more sensitive to the influence of individual data points and is more likely to classify new data points based on the class of its closest neighbor, even if the model may be more prone to overfitting, as it may be too closely influenced by the specific characteristics of the training data. However, since we have selected very discriminant features and we dispose of a large and diverse dataset, we believe the overfitting effect will not be really high. To visualize the effect of increasing k on the performance of our model, we plotted in figure 6 the accuracy and FNR as a function of K. As can be seen in the figure, the performance of the model decreases slightly as K increases. This is to be expected, as increasing K allows the model to consider more neighbors, which can introduce more noise and reduce the model's ability to accurately classify samples.

K	Accuracy	FNR
1	99.67%	0.82%

Table 3: KNN - results on training set

6.2 Performances on the test set

K	Accuracy	FNR
1	99.68%	0.36%

Table 4: KNN - results on test set

We also obtained excellent results on the test set. We could explain the fact that $K = 1$ is the best model parameter also for the test set by noticing that, despite the dataset is made of PDFs that are unique, PDFId is able to extract such discriminant features that different PDFs result to be equal in terms of the extracted features vector. Having overlapped samples in the features space is really helpful for the way KNN classifies because the nearest neighbor of test sample will probably consists in different overlapped samples, most of them belonging to the same class.

Our results demonstrate that KNN is an effective technique for detecting malicious PDFs, and that it can achieve high levels of accuracy and precision with relatively little tuning.

7 FINAL RESULTS

8 ATTACKING THE CLASSIFIERS

In this section, we will present two evasion attacks against the classifiers we built. We are going to increase the number of objects in malicious PDFs so that they will be detected as clean by the classifiers (still preserving their malicious behaviour). (removing is not good).

8.1 Attack against SVM

We have studied an evasion attack based on the gradient descent algorithm. Given a feature vector x characterizing a malicious PDF correctly classified, our goal is to find a x' that will be detected as benign. The constraint is that x' is obtained with a minimal perturbation over x. The function to be minimized is the L1 distance between x' and x:

$$\|x - x'\| = \sum_{i=1}^{d} x_i - x'_i$$

The gradient descent attack will converge to the minimum iteratively. Following the approach in (citare paper kernel svm attack) the feature vector will be moved in the negative direction of the gradient, that means that the gradient descent update step will be:

$$x^{t+1} = x_t - \mu_t \nabla_x f_\Phi(x_t)$$

By recovering that $w = \nabla_x(w^T x_t + w_0)$ for a linear SVM it can be derived that $w = \nabla_x f_\Phi(x_t)$ in a kernel SVM. We can thus obtaing the expression for a RBF kernel SVM from the dual problem definition and properties:

$$\nabla_x f_\Phi(x_t) = \sum_{i \in SV} \alpha_i z_i \nabla_x k(x_i, x_t) \text{ where}$$
$$\nabla_x k(x_i, x_t) = \nabla_{x_t} e^{-\gamma\|x_i - x_t\|^2} = -2\gamma\alpha_i z_i (x_i - x_t) e^{-\gamma\|x_i - x_t\|^2}$$

To better get the idea behind the attack we trained a RBF SVM ($C = 1, \gamma = 0.1$) o the dataset preprocessing it with PCA(with 2 components) just to visualize it. In 7 a random malicious PDF is perturbated iteratively (using $\mu = 0.03$) and it is successfully attacked. We only inspected the working mechanism of the evasion attack by trying to succesdsfully craft benign PDFs from malicious ones.

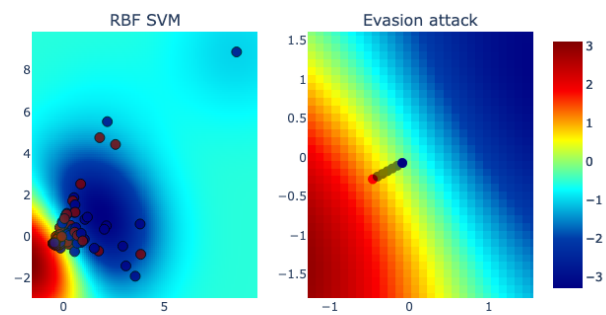

Figure 7: Decision boundary of RBF SVM($C = 1, \gamma = 0.1$) on PCA=2 dataset (left), gradient descent attack on a random malicious PDF (right). (Red: malicious, Blue: benign)

9 CONCLUSIONS AND FUTURE WORKS

10 CONTRIBUTIONS

REFERENCES